Projet S4 MIASHS

La Plateforme du Python en Licence MIASHS

Rapport d'activité

Alexandre CHOLAT, Thibaud DELUCINGE, Adodo Joyce LAMA, Amelia MAHDJOUB

10/05/2025

Engagement de non-plagiat

Nous soussigné.e.s, Alexandre CHOLAT, Thibaud DELUCINGE, Adodo Joyce LAMA, Amelia MAHDJOUB, étudiant·e·s en licence MIASHS, déclarons sur l'honneur que ce document ne comporte aucun plagiat intégral ou partiel de ressources publiées sur toute plateforme de support ;

• Son contenu (textes, images, graphiques ...) cite toutes les sources utilisées pour sa rédaction et uniquement celles consultées ;

• Il reflète le travail personnel de chacun des membres signataires à parts égales.

Nous sommes pleinement conscient.es que le non-respect de cet engagement nous rend passibles de poursuites devant la commission disciplinaire de l'UGA, voire devant les tribunaux de la République française. Nous autorisons l'analyse de tous les documents fournis à l'aide du logiciel compilatio.net avec lequel ils seront archivés.

Fait à Saint Martin d'Hères, le 05/05/2025.

Alexandre CHOLAT Thibaud DELUCINGE Adodo Joyce LAMA Amelia MAHDJOUB

Résumé

Dans le cadre de notre formation, nous avons collaboré sur un projet qui répond à la demande d'un client, visant à proposer aux enseignant.e.s et étudiant·e·s de l'UE Informatique de MIASHS une vision pour l'enseignement du langage Python. Cela a engendré une plaidoirie destinée à la présentation aux responsables de la licence, le recueil de données sur la licence et le statu quo du domaine, la conception d'un site web servant de plateforme d'apprentissage interactive, et la création des cours par modalité innovante. L'élaboration de ce projet a requis une mise en application des pratiques de gestion de tâches, de communication efficace en interne et avec le client, de travail asynchrone coordonné, tout en suivant la *méthode agile* de gestion de projet et programmation pour répondre aux besoins identifiés, assurant la qualité du produit final.

Descripteurs

Python, plateforme d'apprentissage, ressources opérationnelles.

Table des matières

Engagement de non-plagiat.....0
Résumé.....1
Descripteurs.....1
Table des matières.....2
Introduction.....3
Présentation de la plateforme.....4
Mission 0 : Définition de fonctionnement et organisation.....7
Mission 1: Concrétisation et proposition de solution.....8
Mission 2 : Récolte de données.....8
Mission 3: Création du site web.....9
Mission 4: Création de Jupyter Notebooks.....12
Mission 5: Création du plaidoyer.....13
Bilan générale de la gestion de projet.....14
Points faibles.....14
Points forts.....15
Leçons et apprentissages.....15
Conclusion et perspectives.....16
Limitations du travail.....16
Ouvertures.....16
Conclusion.....17
Références.....17
Références consultées.....17
Outils.....17
Annexes.....17
Annexe 1: Glossaire.....17
Annexe 2: Echéancier.....19
Annexe 3: Kanban Board.....20
Annexe 4: Sondage des étudiant·e·s quantitatif.....20
Annexe 5: Sondage des étudiant·e·s qualitatif.....24
Annexe 6: Questions d'entretiens qualitatifs des professeurs d'informatique MIASHS....25
Annexe 7: Sondage des étudiant.e.s en Master.....25

Introduction

La License MIASHS propose actuellement 3 unités d'enseignement (UE), une desquelles est une UE d'informatique, consistant à de l'algorithmique et de la programmation. Au semestres 1,3, et 4, la programmation est dans le langage Java, et au autres semestres différents paradigmes et outils de programmation sont abordés en d'autres langues variées.

Cependant, le choix de Java comme langage de programmation principale commence à produire débat.

Premièrement, l'apprentissage de Java demande l'intégration et mémorisation d'une syntaxe qui est lourde et complexe, constituant une importante barrière d'entrée, surtout pour des profils n'ayant jamais fait de programmation.

Deuxièmement, la pertinence de Java pour la formation des étudiant·e·s mérite réflexion. Alors que le domaine d'application de Java est vaste et offre énormément de possibilités, relativement à d'autres langages, il reste moins utilisé dans les domaines de recherche et d'application. Selon les données de GitHub de 2024, Python est devenu le langage le plus utilisé sur la plateforme, surpassant JavaScript. Parallèlement, l'index de popularité des langages de programmation TIOBE de février 2025 classe Python en première position avec une part de 23.88 %, tandis que la popularité de Java a chuté de 60% depuis 2001.

Parallèlement au débat autour du choix du langage de programmation, une friction pédagogique a été remarquée. Actuellement, les cours magistraux sont présentés sur diapos. Les professeurs sont obligés de changer constamment entre les diapos, tableaux et environnement de développement pour illustrer leurs concepts. Cela est à la fois redondant, une perte de temps, et crée une certaine confusion pour l'apprentissage.

C'est en réponse à ces besoins que notre client nous a proposé d'orienter notre travail de gestion de projet vers une plaidoirie pour Python, avec le but de convaincre l'équipe pédagogique de la valeur que Python apporterait à la licence.

Pour répondre aux demandes de notre client, nous défendons une nouvelle vision de l'apprentissage de l'informatique dans la Licence MIASHS par présentation des résultats d'études et évaluations de la population ciblée, et par création d'un site web pédagogique contenant des cours interactifs en forme de Jupyter Notebooks qui permettent d'introduire une *gamification* ludique et favoriser l'apprentissage des étudiant·e·s.

Présentation de la plateforme

Nous vous expliquons ici les spécificités et fonctions du support pédagogique. Celui-ci prend la forme d'un site web, disponible et clonable depuis son répertoire sur GitHub via le lien suivant: 'https://thibauddevx.github.io/cours_python_projet/'. Le site web intègre des supports de cours interactifs sous la forme de Jupyter Notebooks, des ressources et références, et se différencie des plateformes telles que Moodle par des fonctionnalités de gamification.

FIGURE 1 : PAGE D'ACCUEIL

Notre site maintient la charte graphique de l'UGA en bar de navigation, composée du logo UGA, cliquable pour diriger vers le site de l'université, ainsi que l'affichage des trois onglets de navigation vers les pages d'accueil, de cours, et de ressources. La page d'accueil propose un guidage ergonomique vers les cours, et un compteur de *streaks* (compteur de jours de connexions consécutives) pour motiver les étudiant·e·s et encourager l'assiduité.

FIGURE 2 : PAGE COURS

La page de cours contient un index des cours permettant la navigation entre les différents modules et chapitres. Les cours sont cliquables, déroulant un menu contenant le Jupyter Notebook et des ressources pertinentes.

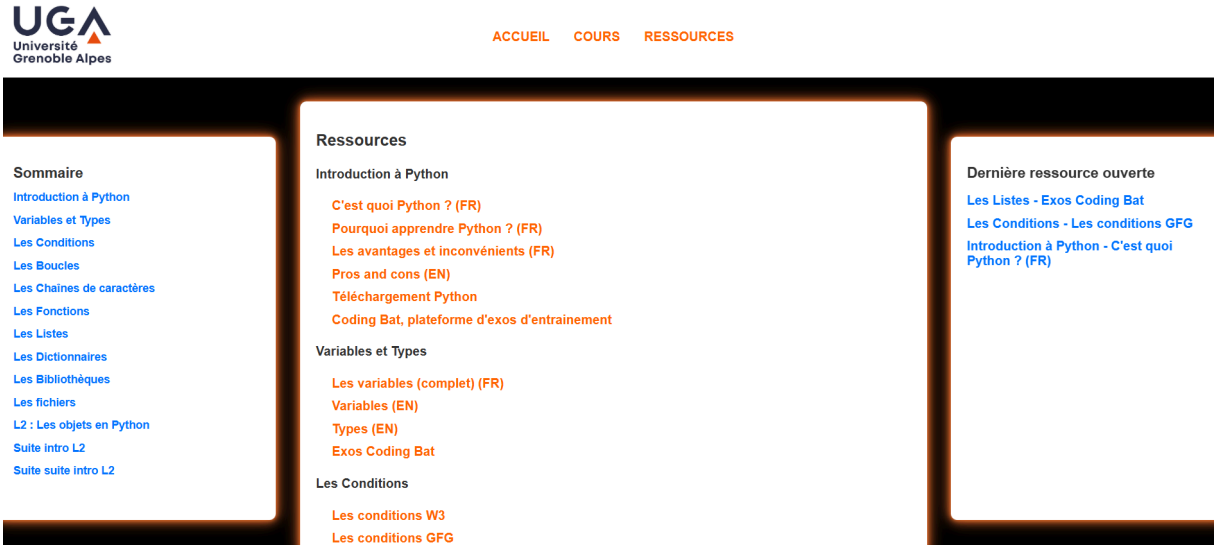


FIGURE 3 : PAGE RESSOURCES

La page ressources est divisée en trois parties. Au centre, un premier index toutes ressources associé à chaque cours. À gauche, un sommaire permet un guidage fluide vers les ressources des cours, tandis qu'à droite un historique des dernières ressources ouvertes garde une trace des références consultées par l'utilisateur. Tous les liens cliquables ouvrent le contenu dans un nouvel onglet de navigation.

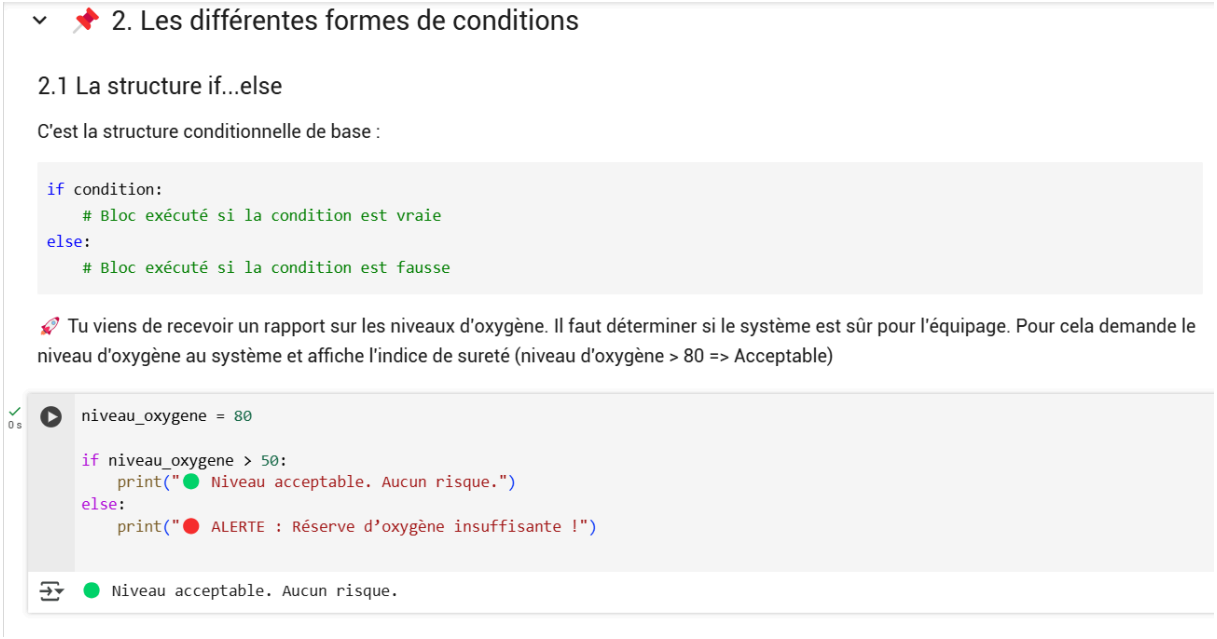


FIGURE 4 : JUPYTER NOTEBOOK

Une fois un cours sélectionné depuis la page web, le Jupyter Notebook est ouvert dans un Google Colab, créant une session de computation sur cloud, et permettant aux enseignants et étudiant·e·s de présenter et suivre un cours multimédia, d'écrire, de modifier, et exécuter du code directement depuis le navigateur web.

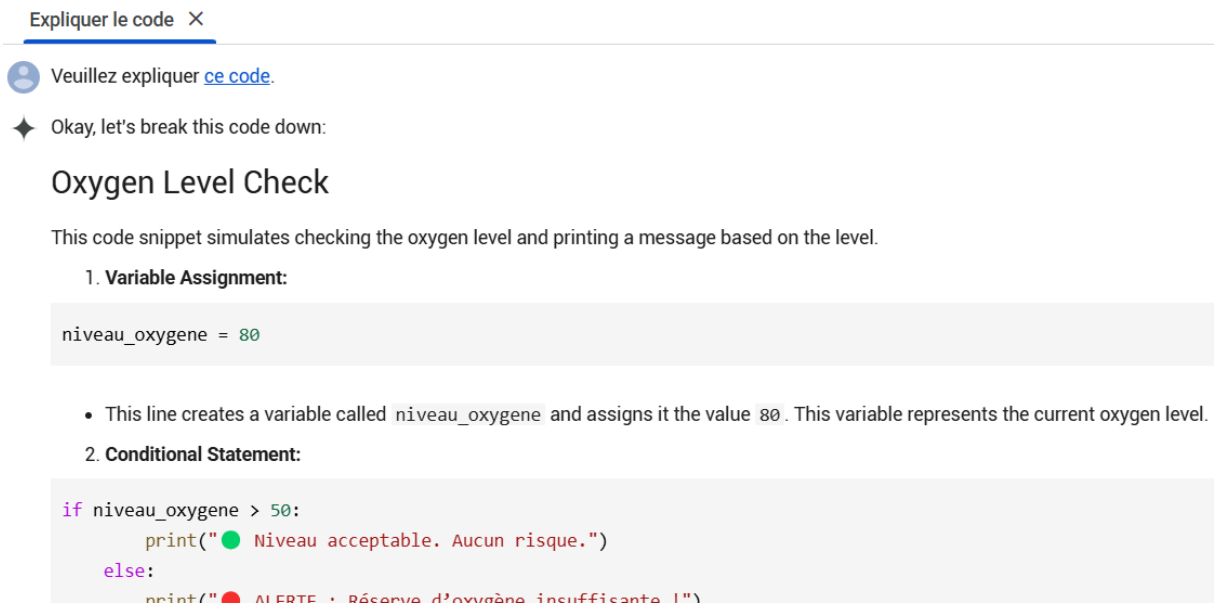


FIGURE 5 : EXPLICATION DE CODE PAR GEMINI

Google Colab est effectivement un IDE python sur navigateur web. L'IA Gemini dans l'environnement permet à l'utilisateur de se faire expliquer le code, servant de tuteur intégré.

Les avantages de cette plateforme et des Notebooks sont multiples. Les étudiant·e·s peuvent coder directement depuis leur navigateur, depuis leur PC ou mobile, et enregistrer leur travail sur cloud pour le modifier plus tard. L'apprentissage interactif, les animations, et les aspects de *gamification* permettent de faciliter l'apprentissage et d'améliorer la rétention des connaissances. Sur le plan pédagogique, les supports dynamiques facilitent la compréhension des concepts informatiques et la démonstration des concepts en temps réel.

Mission 0 : Définition de fonctionnement et organisation

Posteriori au commencement du projet, nous avons commencé par s'approprier une semaine de réflexions et de recherches personnelles pour explorer les outils et méthodologies de gestion de projet. Bien que nous ayons établi des bases et des règles, nos méthodes et stratégies de gestion de projet n'ont pas été les mêmes du début à la fin. Nous nous sommes rapidement rendus compte que certaines pratiques n'étaient pas efficaces et d'autres processus étaient manquants. Nos choix initiaux de gestion de projet ont été remis en question et soumis à de multiples discussions et affinages. Suite à cette évolution non linéaire, notre organisation et mode de fonctionnement final est le suivant.

Premièrement, nous nous sommes dès le début inspiré de la méthode de gestion de projet *agile*. Nous ne voulions pas répartir les tâches et diviser le travail de manière rigide à l'avance, préférant faire des points hebdomadaires ou bihebdomadaires, afin de pouvoir rapidement et efficacement s'adapter aux obstacles et changements. Cette méthodologie a également permis de développer et de tester en continu, facilitant une gestion de temps efficace grâce à un travail autonome.

Pour adopter la gestion de projet *agile* et des méthodes de travail autonomes, il nous a fallu mettre en place des canaux de communication asynchrones et robustes. Dans l'équipe, nous utilisons

WhatsApp pour l'organisation et planification, email pour la distribution de comptes rendus, et la suite Google Workspace pour la collaboration des documents, tableaux, et sondages numériques. Pour la communication asynchrone avec le client, nous privilégions Discord pour échanger des messages et l'espace Cloud UGA pour le partage et annotation des documents.

La communication synchronisée aussi a mérité réflexion. Pour les rendez-vous d'équipe à distance, nous nous servions de Google Meet, avec l'outil Fireflies AI pour automatiser la prise des notes et la rédaction de bilans et listes d'actions. Notre choix d'agilité nous a permis aussi d'échanger spontanément en semaine grâce au fait que nous partagions un lieu d'étude. Les rendez-vous avec client se sont effectués sur son lieu de travail, organisés de fréquence variable selon le stade du projet, un autre avantage de la *méthode agile*.

Pendant les moments de communication synchronisée, nous avions mis en place le *time-blocking* pour nous aider à gérer notre temps, et limiter des longues réunions. Le *time-blocking* consiste à diviser les réunions en blocs de temps prédéterminés, dédiés à des sujets précis. Pour faire cela, nous définissons un ordre du jour à chaque réunion, et le time blocking a permis d'éviter les tangents contre productifs.

La nécessité pour un système de stockage et affichage de tâches et progression de chaque membre nous est rapidement apparu essentiel. Nous avions besoin de pouvoir consulter les informations de progression des autres membres en temps réel. Pour répondre à ce besoin, nous avons adopté le principe du *Kanban board*. Ceci est une sorte d'échéancier dynamique, catégorisant les tâches selon un statut de complétion. Cependant, nous n'avons trouvé aucun logiciel qui répondait à nos besoins. Nous avons donc développé un système qui y répondait, dont un aperçu est disponible en annexe 3. Celui-ci avait comme avantage de s'intégrer directement dans notre Google Workspace, permettant d'automatiser la notification de progression des tâches en alertant les autres membres du groupe quand une tâche était faite et demandait donc un retour.

Dernièrement, nous avons utilisé GitHub et VS Code pour le développement collaboratif du site web et des Jupyter Notebooks.

Mission 1 : Concrétisation et proposition de solution

Après la définition de fonctionnement de gestion et organisation, nous avons commencé à réfléchir aux solutions pour répondre à la problématique. Un premier rendez-vous d'échange d'idées dans lequel nous avions partagé nos propositions individuelles nous a permis de concrétiser une proposition de solution. Nous avons présenté celle-ci au client ainsi qu'un prototype fonctionnel du site web. Suite à la validation de cette solution, nous avons établi un échéancier prévisionnel et les missions suivantes pour répondre au besoin du client du mieux possible. Un cahier des charges a été proposé et itéré tout au long de plusieurs semaines pour aligner notre production de solution au plus près des demandes du client. Chaque itération a été communiquée au professeur via la plateforme du Cloud UGA, et des modifications ont été effectuées à partir des commentaires et suggestions laissés de manière asynchrone sur le document.

Mission 2 : Récolte de données

Notre seconde mission consistait à recueillir des informations pour défendre le rôle de Python dans la licence MIASHS, et de valider l'intérêt de notre solution. Pour établir une argumentation en faveur de Python complète, il était pertinent de recueillir et d'analyser les avis des professeurs et des étudiant·e·s. De plus, les données demandées par notre client, sur les formations informatiques des étudiant·e·s, n'existaient pas. Il nous a été nécessaire de faire des entretiens et sondages pour étudier notre population.

Premièrement, des entretiens qualitatifs, dont les questions sont disponibles en annexe 6, ont été effectués avec deux enseignants d'informatique en MIASHS, avec l'objectif de valider le besoin et notre solution. Ceux-ci ont été effectués en présentiel, à partir d'une brève liste de cinq questions prédéfinies. Grâce aux retours de ces experts, nous avons modifié notre solution initiale, éliminant l'élément de compétition, ainsi qu'évitant du contenu de cours avec de profondes applications aux sciences cognitives ou économie. Nous nous sommes adaptés dans notre proposition, se focalisant sur un produit qui permet aux étudiant·e·s de maîtriser les bases de la programmation, intégrant des aspects ludiques de la gamification, à travers la modalité d'un Jupyter Notebook.

Ensuite, une étude quantitative des étudiant·e·s de la licence a été lancée. Le but de cette étude était de quantifier le bagage informatique des étudiant·e·s lors de leur arrivée en MIASHS, leurs expériences d'apprentissage et utilisation de Python et Java, la distribution des parcours d'origine (bac général, bac technologique, réorientation, etc.), et leur intérêt pour Python. Nous avons choisi de poser ces questions, disponibles en annexe 4, dans l'hypothèse de pouvoir quantifier l'envie des étudiant·e·s pour ce langage et démontrer son utilisation.

Pour distribuer ce sondage, nous avons choisi de le distribuer par email grâce à un communiqué officiel par la secrétaire de la licence. Bien que ce soit un échantillonnage de convenance, nous avons choisi cette méthode d'échantillonnage, car notre population ciblée, les étudiant·e·s de la licence MIASHS, est très petite, et donc relativement peu de réponses représentent une grande part de la population. En effet, cette méthode nous a permis de recueillir 78 réponses, un nombre suffisamment grand pour nous permettre de tirer des conclusions statistiquement significatives.

Grâce à ce taux de réponse élevé, l'étude quantitative a relevé d'importantes informations statistiquement significatives qui nous ont servi pour les diapos de présentation.

Dernièrement, une étude qualitative a suivi pour compléter ces informations, afin d'identifier des tendances aux obstacles et freins à l'apprentissage de la programmation, et déterminer les envies des étudiant·e·s pour apprendre Python vs Java dans un contexte professionnel. Les questions posées sont disponibles en annexe 5.

De la même manière, les anciens étudiant·e·s de MIASHS, actuellement en masters variés, ont été sondés, dans le but de comparer leurs expériences avec les deux langues de programmation et leur pertinence au futur. Leurs questions sont disponibles en annexe 7.

Mission 3 : Création du site web

Notre projet nous offrant une grande liberté, nous avons pu au fil du temps et de nos idées implémenter de nouveaux éléments autours du cours et de la plaidoirie concernant Python. Afin d'organiser tous ces blocs qui forment en leur tout notre projet, nous avons jugé pertinent de passer par l'intermédiaire d'un site web pour exposer nos productions de manière organisée et réfléchie. Cela nous permet de proposer un environnement de travail différent de Moodle en mettant à disposition des élèves un site web spécialisé et dédié à leur apprentissage de Python tout en

implémentant des éléments comme un compteur de jours de connexions consécutives afin de motiver les élèves à la manière de Duolingo.

Concernant la partie technique de la création du site web, il a été construit en partant de zéro avec une base de HTML et CSS. Nous avons également utilisé du JavaScript pour le compteur de jours de connexions consécutives ainsi que pour l'automatisation de la mise à jour du site : les données comme les cours et les ressources sont synchronisées à un fichier JSON afin de permettre une manipulation simples de ces dernières, ainsi ajouter/supprimer des cours/ressources et simple et le site se met à jour de façon automatique. La totalité de la construction du site web s'est déroulée en utilisant GitHub pour permettre à tous les membres du groupe de travailler dessus en simultané et aussi de pouvoir partager notre avancée en temps réel avec le professeur.

Les Jupyter Notebooks sont utilisables via Google Colab, un IDE Jupyter Notebook en ligne nous servant de serveur distant pour faire fonctionner du python, les avantages sont les suivants : gratuit, rapide et proposant de nombreuses bibliothèques python nativement. Nous avons essayé d'autres IDE en ligne, mais aucun ne permet d'atteindre une telle rapidité, et ce, de manière stable, de plus la possibilité de stocker les Jupyter Notebooks directement sur Google Drive et un atout pour les étudiant·e·s qui souhaitent enregistrer leur travail sur internet pour y avoir accès de manière simple.

Les visuels du site internet sont disponibles au début du rapport dans la section 'Présentation de la plateforme', vous y trouverez les différentes pages accompagnées de leur fonctionnement.

Pour ce qui est du déploiement du site web, nous avons opté pour l'utilisation de GitHub Pages, nous assurant un déploiement simple, stable et fiable. Il nous est ainsi très simple de le partager aux élèves grâce à un lien fourni par GitHub. Ainsi, envoyer un email aux étudiant·e·s avec ce simple lien leur permettra d'avoir un accès complet au site web (et à ses futures mises à jour).

Mission 4 : Création de Jupyter Notebooks

Si Python devait intégrer la licence MIASHS, nous envisagerions que les cours se fassent en parallèle à ceux de Java, c'est-à-dire du semestre un au trois, en abordant les mêmes notions.

Nous avons choisi d'utiliser les Jupyter Notebooks, déjà très utilisés dans le monde professionnel, ils offrent un environnement de développement interactif (IDE) qui permet de joindre du code Python (exécutable) et du texte (en Markdown) dans un même document (notebook). Les Jupyter Notebooks sont particulièrement adaptés au projet car ils nous permettent de créer un cours interactif et immersif (l'immersion est abordée plus bas). Les étudiant·e·s peuvent ainsi suivre le cours tout en testant directement les exemples et en répondant aux exercices sans avoir à changer d'outil. Pour les professeurs, les Jupyter Notebooks permettent également de présenter du contenu clair et structuré sans avoir à jongler entre le diaporama et un interpréteur. Les Jupyter Notebooks sont donc des supports complets pour les cours et l'exécution de code.

Le contenu des Jupyter Notebooks est directement inspiré du cours : *Java Initiation à l'informatique et à l'algorithmique* dispensé par Mr. Benoît Lemaire et de 'Algorithmique et programmation par objets' présenté par Mr. Jérôme David. Ce choix s'explique par le fait que nos connaissances actuelles en Python ne sont pas suffisantes pour concevoir intégralement un cours. Les CM de Java ont été une référence concernant les concepts fondamentaux à enseigner ainsi que la progression à suivre.

De nombreuses ressources web ont été engagées lors de notre apprentissage personnel de Python afin de proposer un cours fait par les étudiant·e·s pour les étudiant·e·s, tout en conservant une

structure académique classique en se basant sur les cours de Java dispensés en première année. Naturellement, le cours de Python dévie de ceux en Java quand il s'agit de l'apprentissage de concept plus exclusif à Python.

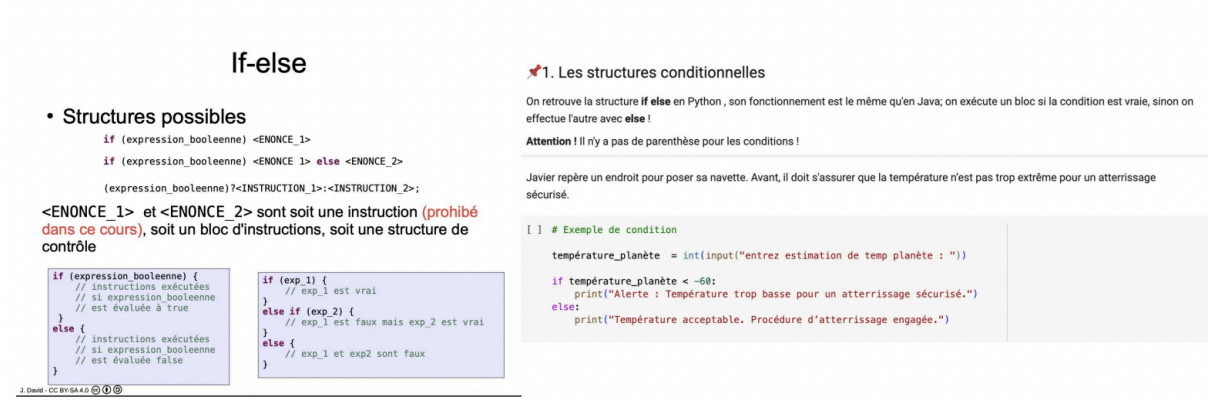


FIGURE 9 : COMPARAISON CM JAVA ET JUPYTER NOTEBOOK

Afin de mettre en avant un apprentissage participatif, chaque notion théorique est accompagnée d'un exemple exécutable que l'étudiant·e peut modifier ainsi qu'un exercice permettant de mettre en pratique ce qu'il a appris précédemment.

À la fin de chaque Jupiter Notebook (correspondant à un cours) un exercice final reprenant ce qui a été vu est proposé pour vérifier la compréhension globale de notions abordées et les appliquer à des situations réelles.

Étant nous-mêmes étudiant·e, nous savons qu'il est parfois difficile de suivre un CM, le passage des cours théoriques à la pratique peut sembler difficile, et c'est pour cela que la gamification est un moyen efficace pour l'apprentissage d'un nouveau langage.

Notre but étant de faire des cours de Python, nous avons décidé de faire de notre possible pour les rendre intéressants aux yeux des étudiant·e·s; une idée a émergé: 'emballer' les cours dans une histoire permettant alors aux étudiant·e·s d'être immergé·e·s et impliqué·e·s dans ces derniers. Les exercices et les exemples sont tous construits autour de cette histoire, le but étant d'augmenter l'implication des élèves dans les cours, cela donne un sens à la progression et suscite la curiosité des étudiant·e·s. L'apprentissage est plus plaisant et rend les concepts abstraits plus concrets.

Mission 5 : Création du plaidoyer

Comme nous l'avons dit plus haut, on nous a aussi confié la mission de défendre l'idée d'intégrer le langage Python dans le programme de la licence MIASHS. Cette tâche s'est montrée plus compliquée que prévu, car elle nous a conduit.e.s à problématiser notre démarche, puis à élaborer une argumentation reposant sur une réflexion pédagogique, technique et institutionnelle.

Dès les premières réunions, nous avons rencontré plusieurs obstacles. En effet, la question n'était pas seulement "Faut-il intégrer Python ?", mais plutôt "Comment l'intégrer intelligemment ?". Devait-on ajouter une nouvelle matière au programme, réorganiser les enseignements existants, ou bien remplacer un ou plusieurs langages déjà présents ? Ces questions ont mis en lumière une diversité d'opinions au sein du groupe. Certain.e.s considéraient le fait de faire l'intégralité du programme d'Informatique avec du Python, tandis que d'autres considéraient une approche plus modérée, un remplacement partiel. Ces dernier.ère.s soulignaient l'intérêt pédagogique d'avoir plusieurs langages de programmation, ce qui est un atout à ne pas négliger. Malgré nos échanges, aucune solution ne faisait l'unanimité, ni sur le langage à écarter, ni sur la façon précise d'introduire Python.

Nous avons décidé de prendre une approche plus pragmatique. Plutôt que de choisir entre "remplacer" ou "ajouter", on a choisi de mettre en avant la flexibilité de Python, sa capacité à s'adapter au programme existant. Nous avons ainsi montré qu'il permettait d'enseigner les paradigmes fondamentaux de la programmation (impératif, fonctionnel, orienté objet). Cette démarche nous a permis d'éviter notre dilemme et de proposer une intégration partie par partie, laissant de la liberté aux enseignant.e.s.

Dans un premier temps, notre plaidoyer s'est concentré sur les avantages connus de Python : une syntaxe simple et lisible, une grande popularité dans les mondes académiques et professionnels, une large bibliothèque et une communauté active. Cependant, nous nous sommes rendues compte que cette démarche était insuffisante, car elle ne prenait pas en compte le contexte de la licence MIASHS. Suite aux recommandations de notre commanditaire, nous avons réajusté notre approche en intégrant des éléments en lien avec la licence MIASHS.

Nous avons donc ajusté notre approche pour prendre en compte les attentes et les réalités des enseignant.e.s comme des étudiant·e·s. Cet exercice s'est révélé complexe : nous avons dû émettre des choix et parfois remettre en questions nos propres idées, et confronté nos biais. Mais ce travail de fond nous a permis de mieux cerner les enjeux et d'enrichir notre réflexion. Progressivement, nous sommes arrivé.e.s à une conviction partagée : Python a toute sa place dans la formation MIASHS, à condition qu'il soit intégré de manière réfléchie et adaptée.

En continuant sur cette lancée, nous avons décidé de rajouter dans notre argumentation des résultats de plusieurs sondages conduits auprès des étudiant.e.s de la licence MIASHS et de quelques masters accessibles depuis ce parcours. Ces données nous ont aidé à mieux comprendre les besoins, les envies, et aussi les difficultés rencontrées dans l'apprentissage de la programmation. Nous avons remarqué que la majorité des participant.e.s avaient déjà été initié.e.s à Python avant leur entrée à l'université. De plus, de nombreux.ses étudiant·e·s expriment le souhait d'approfondir leur maîtrise de ce langage.

Comme support, nous avons choisi le format du diaporama. C'était, selon nous, la meilleure manière de rendre notre message clair, structuré et abordable. Ce format nous offrait aussi la liberté d'exposer nos idées de manière synthétique, tout en les illustrant. Comme notre présentation était destinée à un public varié (étudiant·e·s, enseignant.e.s, ou responsables pédagogiques), nous avons pris soin d'y inclure une introduction claire, avec les objectifs, les enjeux, et les principales notions abordées. Cette idée, suggérée par notre commanditaire, visait à rendre notre travail compréhensible pour des personnes non spécialistes de l'informatique, sans pour autant perdre en rigueur.

Pour finir, nous avons voulu aller plus loin que nos seuls constats et sondages. Nous avons donc décidé d'analyser les résultats des sondages avec la même réflexion nécessaire pour choisir quel langage de programmation prendre dans notre formation. Plusieurs éléments sont remontés comme

étant essentiels: la clarté de la syntaxe, la cohérence avec le reste des enseignements, et bien sûr, l'adaptation des étudiant.e.s du monde professionnel.

Cette démarche nous a permis d'avoir un regard plus développé sur l'intérêt de Python. À nos yeux, c'est un langage qui coche toutes les cases pour être introduit dès la première année de licence, non pas comme un remplacement brutal, mais comme un complément pertinent aux outils déjà en place. En prenant en compte à la fois la théorie, les retours des étudiant·e·s, et les réalités de la formation, nous avons construit une position claire : intégrer Python progressivement, dès le premier semestre, dans une logique pédagogique cohérente, c'est non seulement faisable, mais surtout bénéfique pour tout le monde.

Bilan général de la gestion de projet

Points faibles

Notre gestion de projet est à améliorer sur deux axes principaux. Le premier axe d'amélioration est celui de la planification de tâches. Plus spécifiquement, lors de la répartition des tâches hebdomadaires, les actions ont été nommées et explicitées, mais les critères de qualités et quantificateurs de réussite n'ont pas été discutés. Ce manque d'attentes claires pour l'exécution du travail avait pour résultat un trop grand nombre de boucles *travail individuel — retour d'équipe*. Cette inefficacité s'est prononcée de manière discrète au long de l'échéance, car les conséquences se manifestent occasionnellement, mais elle représente tout de même une importante opportunité d'amélioration pour des prochains projets, avec le potentiel d'augmenter la qualité des travaux.

Le deuxième axe d'amélioration est celui de la pratique et méthode de retour et critique de travail du groupe. En principe, nous avons discuté de l'importance de ceci lors de la définition de fonctionnement de l'équipe et nous y avons consacré du temps après chaque tâche de l'échéance pour s'assurer qu'il ne manquerait pas de temps pour faire des retours. L'outil de gestion de tâche développé par l'équipe avait même une fonction de notification d'alerte envers l'équipe quand une tâche était marquée « Needs Review ». Cependant, aucune méthode n'a été mise en place pour assurer l'exécution de l'analyse et de la communication de retour, et aucun canaux de communication n'a été défini ou consacré spécifiquement pour cet usage. En pratique, certains travaux n'ont pas été après avoir été marqués comme en besoin de retour. Ce manque de méthodologie s'est manifesté par un faible taux de responsabilité pour assumer le retour envers les autres, et à un niveau plus macroscopique, par un manque de contrôle qualité.

Points forts

Grâce à une culture de respect mutuelle des idées et des contributions des uns et des autres, nous avons fait preuve d'une excellente adaptabilité au long de l'exécution du travail.

Pour commencer, le choix de méthode de gestion *agile* nous a très bien servi. Des réunions hebdomadaires ont permis une distribution de travail relativement homogène tout au long de l'échéance, et produisant comme résultat une parfaite gestion de temps ainsi que des conditions favorables pour un travail de qualité. En maintenant une communication régulière dans l'équipe, et en communiquant efficacement notre progression avec le client, on a pu répondre de manière adaptable aux demandes évolutives de celui-ci, qu'elles soient pour des données et sondages additionnels ou d'autres fonctionnalités et éléments d'utilisation de la plateforme pédagogique.

Deuxièmement, quand nous avions rencontré des frictions et des inefficacités dans notre organisation et fonctionnement, l'équipe a pu identifier les problèmes, rechercher et proposer des solutions, et mettre ces nouvelles solutions en place de manière rapide. Face à un manque de communication sur le progrès des tâches asynchrones, nous avions adopté le *Kanban board*. Cependant, les logiciels de *Kanban board* ne correspondaient pas à nos besoins, et l'équipe s'est de nouveau adaptée pour développer un logiciel qui répondait parfaitement aux problèmes. Face à des réunions qui durent trop longtemps et qui menaient à peu de décisions concrètes, l'équipe a mis en place la méthode du *time-blocking* pour traiter des sujets prédéfinis et augmenter notre productivité. Nous avions également adopté des outils pour automatiser la prise de notes pendant nos réunions pour réduire le temps consacré au secrétariat. Dernièrement, nous avons remarqué un manque de retours généraux sur les contributions de chacun, ce qui empêchait une croissance personnelle en relations humaines et en techniques de gestion de projet. Suite à l'identification de cette lacune, nous avons mis en place un logiciel pour automatiser l'échange des retours pour démocratiser leur échange.

Leçons et apprentissages

Un moment de réflexion sur l'évolution du projet et de notre collaboration nous a relevés plusieurs leçons. Il nous est devenu évident que l'adaptation aux frictions et aux inefficacités a été une capacité essentielle pour notre réussite, par l'adoption de nouveaux outils et méthodes de travail. De même, les bénéfices de la flexibilité de la *méthode agile* pour distribuer le travail au sein de l'équipe et la réduction du stress collectif, a amélioré la qualité de nos échanges et la fréquence de celles avec notre client, ce qui a pour résultat l'augmentation de notre productivité.

Parallèlement, nous avons réalisé l'importance d'expliciter les critères de réussite d'une tâche, ainsi que l'intérêt de faire des contrôles de qualité pour assurer que le travail répond aux critères définis. Pour des futurs projets, nous mettrons les pratiques suivantes en place : premièrement, des canaux de communications seraient à définir pour chaque mission du projet. Deuxièmement, une personne serait préalablement attachée comme responsable de contrôle de qualité à chaque tâche dans l'échéancier, et les critères de qualité qu'elle évalue seront définis clairement suite à la création de l'échéancier.

Enfin, la nature informatique de notre solution a servi d'expérience idéale pour explorer et mettre en pratique différents outils de collaboration. Un apprentissage particulièrement pertinent était notre collaboration sur le développement web utilisant GitHub, utilisant les opérations de push, pull, clone, et autres, pour coordonner notre travail asynchrone. Cette expérience est intéressante, car GitHub est un outil de référence dans les domaines d'*open source* et du développement au plus large.

Conclusion et perspectives

Limitations du travail

Un grand obstacle à la vision initiale a été un blocage de droit d'hébergement issu de la politique universitaire. Il nous a été impossible d'héberger ou de se faire héberger le site web sur les serveurs officiels de l'institution. Bien que le produit final soit disponible publiquement en *open source* sur GitHub, l'implémentation et hébergement de celui-ci sera à la responsabilité de la licence. Les questions de la *scalabilité* de cette solution seront également hors de notre portée, car la

modification du contenu affiché sur le site web demande actuellement une modification directe du code, et insertion des cours et du texte dans le dossier 'Assets' par fichier JSON.

Une autre lacune est le dernier sondage qualitatif qui avait pour objectif d'identifier des tendances aux obstacles et freins à l'apprentissage de la programmation. Finalement, les réponses récoltées n'ont pas pu être exploitées, aucune tendance n'a pu clairement être identifiée. Une tentative de classification de réponses par un modèle de traitement du langage naturel s'est révélée erronée et inefficace. L'erreur a été la suivante : le sondage qualitatif aurait dû être fait avant le sondage quantitatif, afin de pouvoir mieux sélectionner et affiner le choix de données qu'on voulait quantifier.

Dernièrement, le site web et son contenu a uniquement été validé en interne. Des contrôles de qualité ont été faits par l'équipe et un produit final validé par notre client, mais il serait encore plus intéressant de recevoir des retours des utilisateurs réels.

Ouvertures

Un *dashboard* administratif, avec une interface qui permet d'ajouter ou modifier le contenu du site, serait intéressant à envisager. L'avantage serait de permettre aux professeurs la possibilité d'interagir ergonomiquement avec le site, sans devoir modifier directement le contenu du fichier JSON.

Il est aussi à noter de la présence de la fonctionnalité d'IA intégrée dans l'espace Google Collab. En effet, l'IA Gemini dans l'environnement permet à l'utilisateur de se faire expliquer le code. Le futur contenu du cours pourrait être créé intentionnellement dans le but d'être complémenté par cet outil, dans l'optique que l'utilisateur pourrait se servir du tuteur intégré pour pouvoir l'assister dans sa démarche de recherche de solution pour un exercice donné. Les futurs Jupyter Notebooks pourraient donc inclure des exercices plus poussés, sans la nécessité de présenter une seule solution de référence.

Conclusion

Bien que nous avions rencontré des freins et des obstacles, l'adoption de nouveaux outils et une adaptation organisationnelle dans notre gestion et exécution du travail ont témoigné à l'efficacité de la *méthode agile*. Un rythme de travail constant, couplé à une culture de respect mutuelle au sein de l'équipe, s'est prouvé un cadre exceptionnellement fertile pour une collaboration efficace. Nous sommes fiers de conclure que notre travail puisse servir de modèle et référence dans l'élaboration de la discussion du rôle de Python dans la licence MIASHS.

Références

Références consultées

- *Tiobe index*. TIOBE. (2022, June 3). https://www.tiobe.com/tiobe-index/java/
- Staff, G. (2024, October 29). Octoverse: AI leads Python to top language as the number of global developers surges. The GitHub Blog. https://github.blog/news-insights/octoverse/octoverse-2024/
- Wikimedia Foundation. (2025). Wikipedia. Wikimedia Foundation. https://www.wikipedia.org/

Outils

- OpenAI. (2025). ChatGPT. OpenAI. https://chat.openai.com/
- ZeroGPT. (2025). ZeroGPT. https://www.zerogpt.com/
- Citation Machine. (2025). APA Citation Generator. https://www.citationmachine.net/apa
- Microsoft. (2025). Microsoft Copilot. Microsoft
- Microsoft. (2025). Visual Studio Code. Microsoft. https://code.visualstudio.com/
- GitHub, Inc. (2025). GitHub. https://github.com/
- Google. (2025). Google Workspace. Google. https://workspace.google.com
- Mistral AI. (2025). Le Chat. Mistral AI. https://chat.mistral.ai/
- Fireflies AI. (2025). Fireflies AI. https://www.fireflies.ai/
- Discord Inc. (2025). Discord. Discord Inc.
- Language Tool. (2025). Language Tool. https://languagetool.org/
- Université Grenoble Alpes. (2025). Cloud UGA. Université Grenoble Alpes.

Annexes

Annexe 1 : Glossaire

Git : Système de gestion de versions permettant de suivre les modifications du code et de collaborer sur des projets.

Open Source : Logiciels dont le code source est accessible et modifiable par tous.

Scalabilité : Capacité d'un système à s'adapter à une augmentation de la charge de travail (nombre d'utilisateurs, volume de données, etc.).

Gamification : Intégration de mécaniques de jeu (points, badges, défis) dans des contextes non ludiques pour stimuler l'engagement.

Repository : Emplacement de stockage centralisé où le code source d'un projet est géré via Git.

Nudges : Techniques incitatives visant à encourager certains comportements, ici pour motiver les étudiant·e·s à suivre le cours.

Dashboard : Interface graphique permettant de visualiser et de gérer des données ou fonctionnalités en temps réel, souvent utilisée pour le suivi de performances, l'administration de systèmes ou la gestion de contenu.

Open source : Un code conçu pour être accessible au public. N'importe qui peut voir, modifier et distribuer le code à sa convenance.

Méthode Agile : Approche itérative et incrémentale de la gestion de projet, qui met l'accent sur la flexibilité, la collaboration et la satisfaction du client.

Kanban board : Outil visuel de gestion de projet qui permet de suivre le progrès des tâches en les organisant sur un tableau divisé en colonnes

Time-blocking : Technique de gestion du temps qui consiste à diviser une journée ou réunion en blocs de temps dédiés à des tâches ou des types d'activités spécifiques.

Annexe 2 : Échéancier

Tâches	Début	Durée (en h)	Fin	Progression (en %)	Janvier-2025			Février-2025		
Avancement global	13/01/2025	7	15/05/2025		S3	S4	S5	S6	S7	S8
Lancement	13/01/2025		15/05/2025							
Echéancier	13/01/2025		15/05/2025							
Définition des tâches à réaliser	13/01/2025									
Définition des durées par tâches										
Cahier des charges										
Définition du contexte										
Définition de l'objectif du projet										
Définition des contraintes										
Définition des solutions										
Planification										
Rédaction de l'échéancier										
Rédaction esquisse du cahier des charg	05/02/2025		22/02/2025							
Date de la réunion de validation du cahier des charges										
Exécution										
Produire une maquette										
Dossier en faveur de Python	20/01/2025	27/03/2025								
La création des Jupyter Notebooks	05/02/2025									
création des sondages										
Création du site web	05/02/2025									
Réunion de validation finale										
Analyse des failles/problèmes										
Proposition de solutions aux problèmes										
Rédaction du rapport d'activité										
Préparation de la soutenance										
Suivi/Phase test										
Test du produit										
Réévaluation et mise à jour										
Validation par le commanditaire										
Finalisation de l'échéancier final										
Bilan de la gestion du projet										
Clôture du projet										
Livraison du produit/rendre le rapport										
Réunion de clôture de projet										

Annexe 3 : Kanban Board

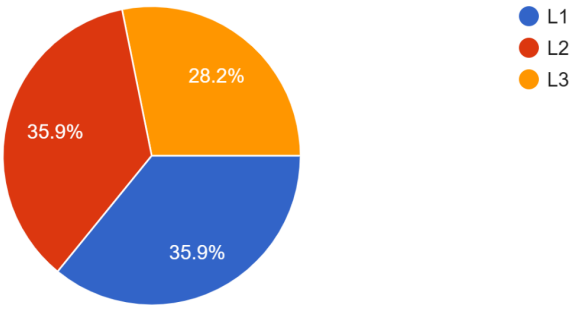
Taches du Projet MIASHS S4

STATUS	PRIORITY	DEADLINE	TASK NAME	ASSIGNEE	DESCRIPTION	Review Notification	Feedback
Sondage							
Complete	Medium	25/3	Traitement de donnees	Alexandre	Trouver la meilleure option de présenter les infos		
Complete	Medium	5/4/2025	Entretiens Qualitatives	Tous	Etudiants de License et Master		
Complete	Medium	10/3	Publication	Joyce	Publication et partge des sondages		
Complete	High	25/3	Questions qualitatives pour étudiants MIASHS	Alexandre	Sondage pour collecte d'informations qualitatives sur les etudiants MIASHS et leur apprentisssage de		pour la question "Pensez-vous qu'un apprentissage pa
Complete	High	7/4	Envoie de Questions qualitatives pour étudiants MIASHS	Joyce			
	High		Traitement de sondage qualitatives sur les avis des		Nous avons un fichier de 17 reponses: nettoyer donees pour extraire reponses?		
Liverables							
Complete	Medium	26/2	CC	Alexandre	Cahier des Charges		
In Progress	High	08/05	Rapport Ecrit	Tous	details d'eval: https://docs.google.com/document/d/1sIRiTSiQ4R03		
Needs Review	Medium	03/04	Rapport: Forme et mise en page	Alexandre	Forme, Engagment non plagiat, Table de matieres, Resume , descripteurs	Notified	
Needs Review	Medium	00/00	Rapport: Introduction	Alexandre	une page?	Notified	
Needs Review	Medium	16/04	Rapport: Présentation du Site	Alexandre		Notified	
Needs Review	Medium	01/05	Rapport: Mission 0 : Définition de fonctionnement et organisation	Alexandre		Notified	
In Progress			Rapport : Mission 5: Création de la plaidoirie	Joyce	diapos		
Needs Review			Rapport: Mission 1	Alexandre		Notified	
Needs Review			Rapport: Mission 2	Alexandre		Notified	
In Progress			Rapport: Mission 3: Création du site web	Thibaud			
In Progress			Rapport: Mission 4: Création de Jupyter Notebooks	Amelia			
Needs Review			Rapport: Points faibles	Alexandre		Notified	

Annexe 4 : Sondage des étudiant·e·s quantitatif

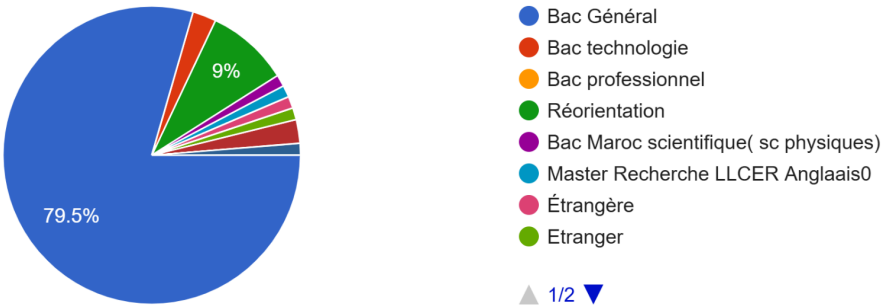
Quelle est votre année actuelle en MIASHS ?

78 responses



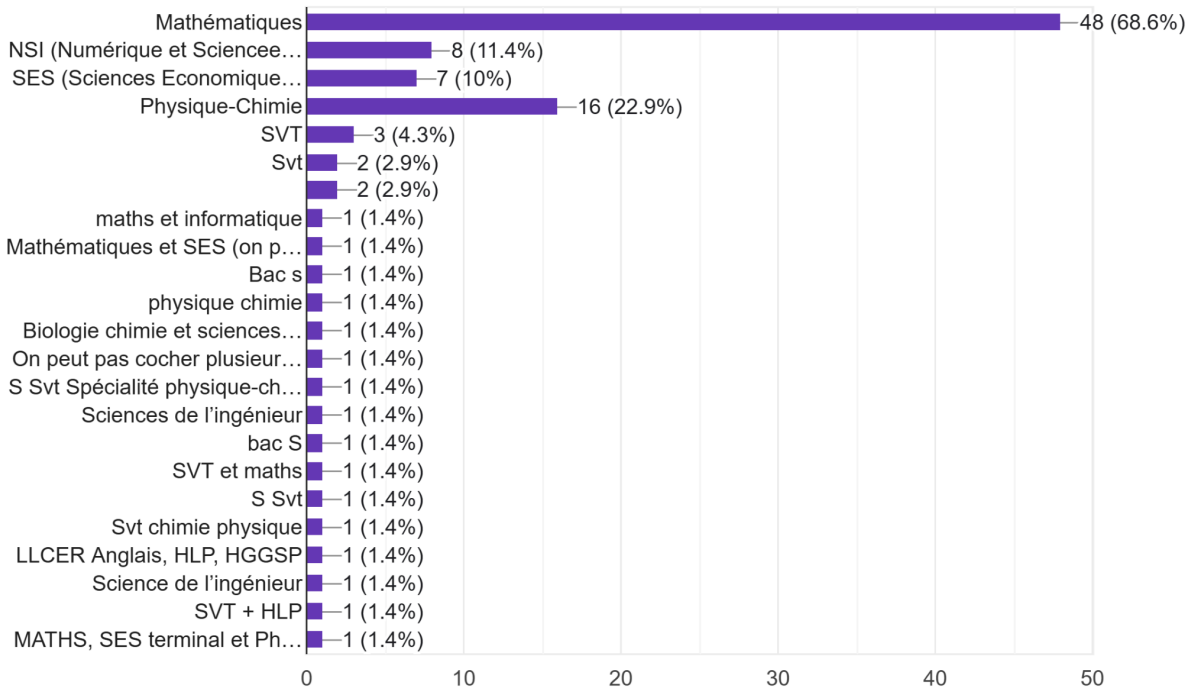
Quel était votre parcours avant d'entrer en licence MIASHS

78 responses



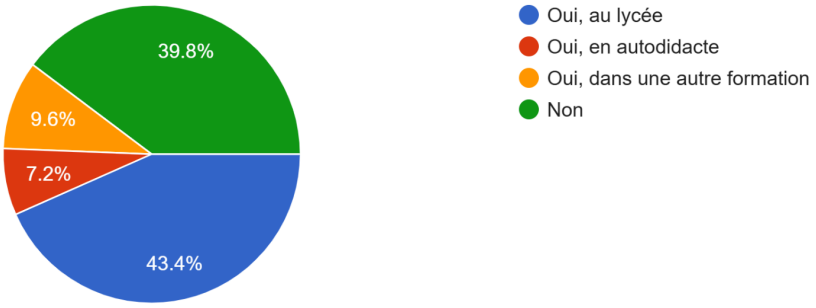
Si vous avez fait un bac général, quelles spécialités avez-vous suivies ?

70 responses



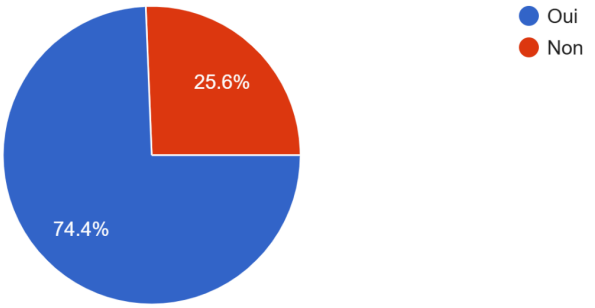
Avez-vous suivi des cours d'informatique avant d'entrer en MIASHS ?

78 responses



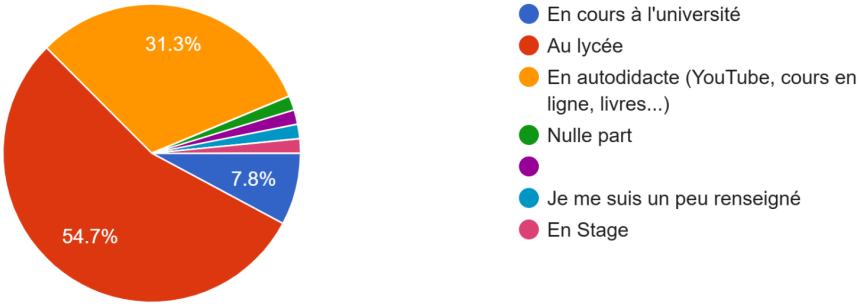
Avez-vous déjà utilisé Python ?

78 responses



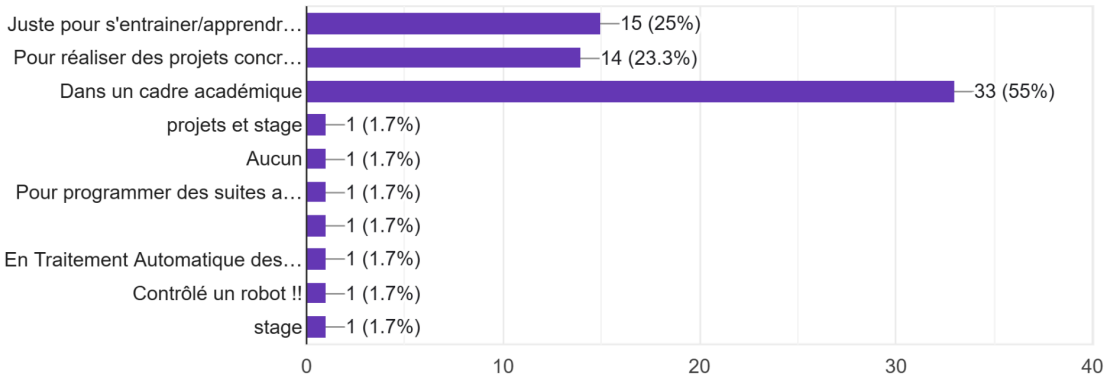
Si oui, comment avez-vous appris Python ?

60 responses



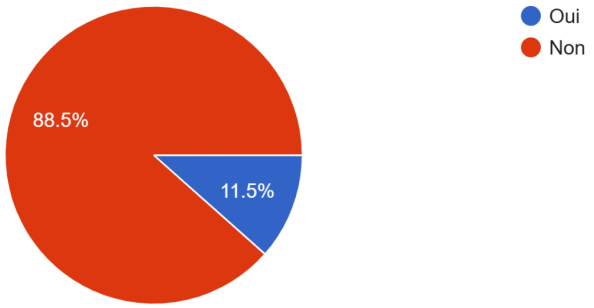
Dans quel cadre avez-vous utilisé Python ?

60 responses



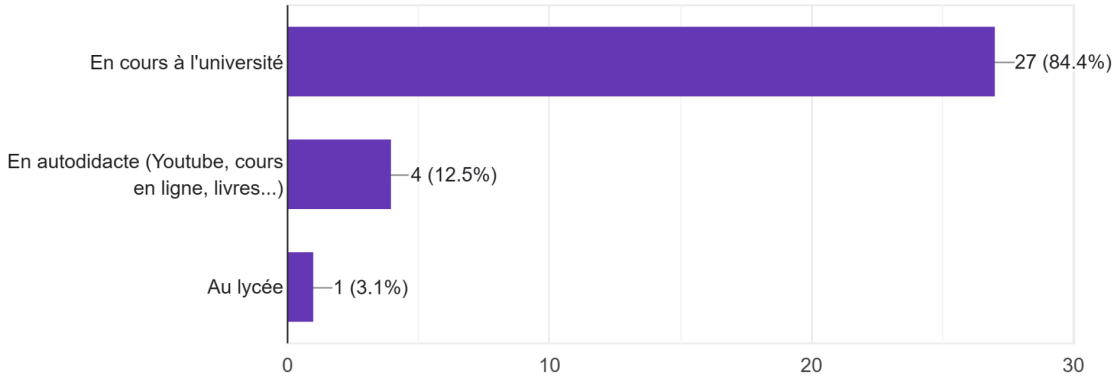
Avez-vous déjà utilisé Java avant la licence MIASHS ?

78 responses



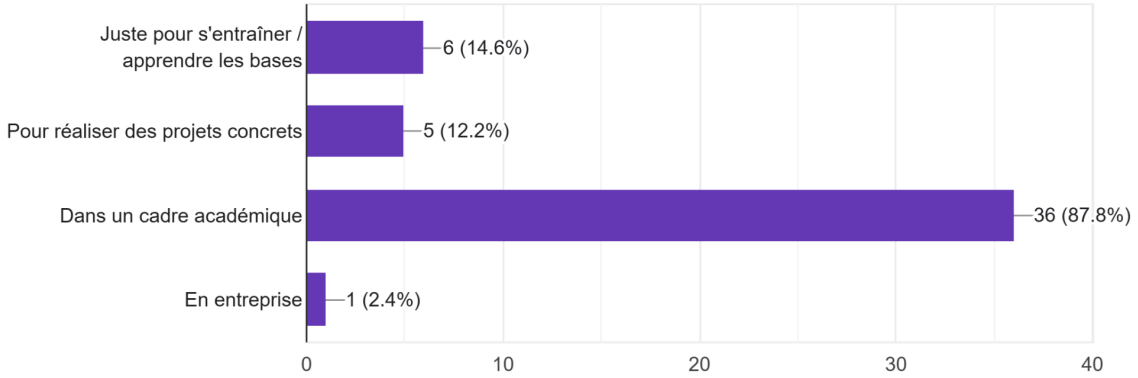
Si oui, comment avez-vous appris Java ?

32 responses

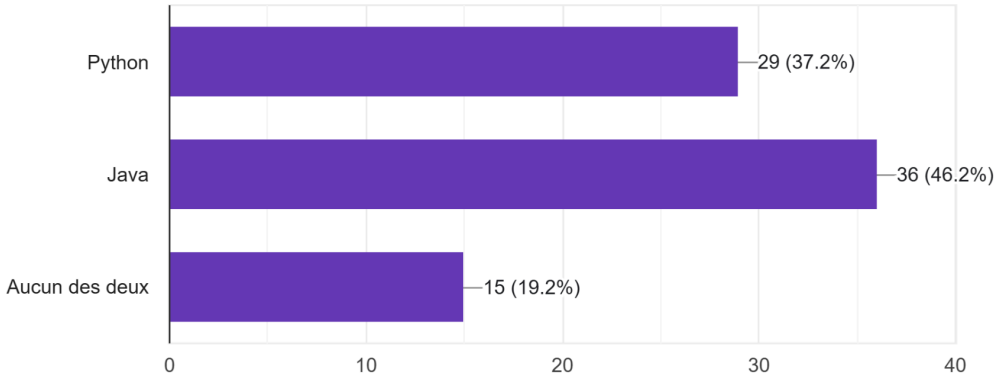


Dans quel cadre avez-vous utilisé Java ?

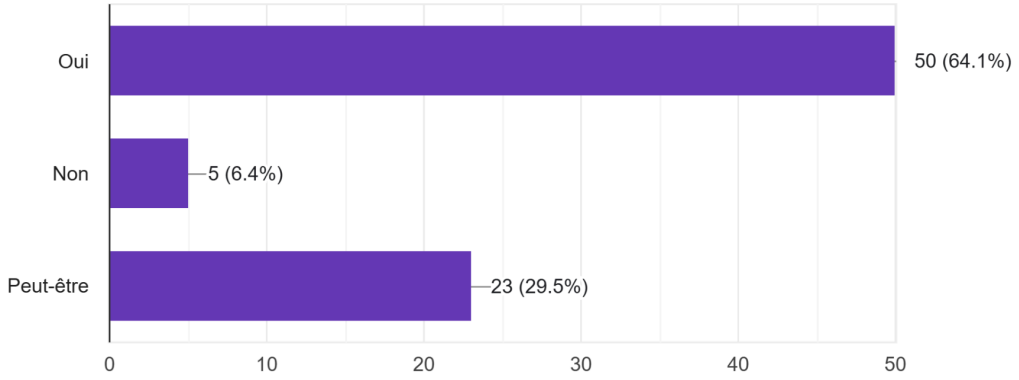
41 responses



Entre Python et Java, lequel préférez-vous ?
78 responses



Avez-vous envie d'approfondir votre apprentissage de Python ?
78 responses



Annexe 5 : Sondage des étudiant·e·s qualitatif

1. Comment s'est passé votre apprentissage du Python ? Facile, difficile ? Pourquoi ?
2. Comment s'est passé votre apprentissage de Java ? Facile, difficile ? Pourquoi ?
3. Si vous avez appris Python et Java, lequel vous semble plus accessible en début de parcours ? Pourquoi ?
4. Que pensez-vous de débuter la programmation avec Python en L1 plutôt qu'avec Java ? Serait-il pertinent d'apprendre les deux dès la première année ?
5. Selon vous, apprendre trois/deux langages différents en L1/L2 serait un avantage ou une difficulté supplémentaire ? Pourquoi ? (Répondez si vous êtes en L2/L3)
6. Entre Java et Python, lequel préférez-vous et pourquoi ? (D'un point de vue personnel)
7. Pensez-vous qu'un apprentissage par le jeu aiderait à mieux comprendre la programmation ?
8. Si vous pouviez choisir votre langue d'apprentissage en L1, que choisiriez-vous entre Java, Python ou un autre langage, et pourquoi ?

Annexe 6 : Questions d'entretiens qualitatifs des professeurs d'informatique MIASHS

1. Connaissez vous les Jupyter Notebooks? Si oui, que pensez-vous de leur valeur dans un contexte pédagogique? Les Jupyter Notebooks sont-ils plus adaptés au CM ?
2. Si le programme de MIASHS était totalement à reconstruire de zéro, pensez-vous qu'il serait intéressant de remplacer un ou les cours de Java par des cours de python?
3. Pensez vous qu'il serait intéressant d'enseigner la programmation dans un contexte éco / sciences cog ?
4. Que pensez- vous de l'idée de gamifier un CM ou un TD ?
5. A votre avis, la compétition entre élèves a-t-elle un rôle dans la conception d'une pédagogie?

Annexe 7 : Sondage des étudiant·e·s en Master

1. Quel est le nom de votre de master et le nom de votre licence ?
2. Comment décririez-vous l'apprentissage du langage de programmation que vous avez appris dans votre licence ?
3. Quel langage de programmation utilisez-vous en licence ?
4. Les compétences acquises en programmation durant la licence vous ont-elles été utiles en master ? Si oui, comment ?
5. Avez-vous dû apprendre un autre langage en master ? Si oui, lequel et pour quelles raisons ?
6. Auriez-vous préféré avoir un autre langage en complément pour faciliter votre transition vers le master ? Si oui, lequel ?
7. Quels langages de programmation sont les plus demandés dans votre domaine aujourd'hui ?
8. Which programming languages are most in demand in your field today?
9. Si vous deviez conseiller un langage aux étudiant·e·s en MIASHS, lequel choisiriez-vous et pourquoi ?

Annexe 8 : Grille d'auto-évaluation

Gec 2.1	Alexandre	Amélia	Joyce	Thibaud
C1-participer	1,55	1,35	1,55	1,55
C2-s'investir	1,7	1,3	1,5	1,5
C3-travail collectif	1,5	1,5	1,5	1,5
TOTAL	4,75	4,15	4,55	4,55
Gec 2.2	Alexandre	Amélia	Joyce	Thibaud
C1-participer	1,5	1,5	1,5	1,5
C2-s'investir	1,7	1,3	1,5	1,5
C3-travail collectif	1,5	1,5	1,5	1,5
TOTAL	4,7	4,3	4,5	4,5
Gec 2.3	Alexandre	Amélia	Joyce	Thibaud
C1-participer	1,5	1,5	1,5	1,5
C2-s'investir	1,7	1,4	1,5	1,4
C3-travail collectif	1,5	1,5	1,5	1,5
TOTAL	4,7	4,4	4,5	4,4
Gec 2.4	Alexandre	Amélia	Joyce	Thibaud
C1-participer	1,4	1,4	1,55	1,65
C2-s'investir	1,8	1,2	1,4	1,6
C3-travail collectif	1,5	1,5	1,5	1,5
TOTAL	4,7	4,1	4,45	4,75
	Alexandre	Amélia	Joyce	Thibaud
moyenne C1	1,4875	1,4375	1,525	1,55
moyenne C2	1,725	1,3	1,475	1,5
moyenne C3	1,5	1,5	1,5	1,5
moyenne globale	1,570833333	1,4125	1,5	1,516666667